

PROJECT - GPS Data Transmission Using MQTT and Real-Time Map Visualization

Experiment: Transmission of simulated GPS coordinates (latitude & longitude) generated in a C program to a local MQTT broker and visualization of the live location updates on a map using a Python subscriber application.

➤ Objective

To generate random GPS-like coordinates using a C program, publish them periodically to an MQTT broker running on a local host, and receive the data using a Python MQTT client. The received coordinates are then plotted in real time on a map, updating every 5 seconds to simulate live location tracking.

➤ Theory

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol widely used in IoT applications for efficient data exchange between devices over a network.

In this experiment:

- A **C program** acts as the MQTT publisher.
- It generates random latitude and longitude values within valid geographical ranges.
- These values are published to a topic on a **local MQTT broker (localhost)**.
- A **Python program** acts as the subscriber using the Paho MQTT client.
- Python receives the messages and plots the coordinates on a map in real time.

The MQTT broker (commonly Mosquitto) handles message routing between publisher and subscriber without direct connection between them. This architecture enables scalable and decoupled communication in IoT systems.

MQTT is standardized under OASIS and designed for low-bandwidth, high-latency environments.

➤ Hardware Used

- Laptop / PC
- C Compiler (GCC / MinGW / Linux GCC)
- MQTT Broker (Eclipse Mosquitto – localhost)
- Python 3
- Paho-MQTT Library
- Matplotlib / Folium (for map visualization)
- Visual Studio Code PySerial Library

➤ Procedure

- Install and start the MQTT broker on the local system (localhost).
- Create a C program to generate random latitude and longitude values within valid ranges:
- Latitude: -90 to +90
- Longitude: -180 to +180
- Initialize MQTT client in C using an MQTT library.
- Connect the C publisher to the local broker.
- Publish GPS data to a topic (e.g., gps/data) every 5 seconds.

- Develop a Python subscriber program using Paho MQTT.
- Subscribe to the same topic (gps/data).
- Receive incoming GPS coordinate messages.
- Parse latitude and longitude values from the message.
- plot the received points dynamically on a map.
- Continuously update the map every 5 seconds to reflect new positions.

➤ Program Operation

```
C publisher.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <windows.h>
4
5  char *locations[] =
6  {
7      "12.9788,77.5996",
8      "12.9763,77.5929",
9      "12.9797,77.5907",
10     "12.9507,77.5848",
11     "12.9756,77.6066",
12     "12.9987,77.5920",
13     "13.0098,77.5511",
14     "12.8008,77.5773",
15     "12.8346,77.4010",
16     "13.3702,77.6835"
17 };
18
19 int main()
20 {
21     int i = 0;
22
23     while(1)
24     {
25         char cmd[200];
26
27         sprintf(cmd,
28             "mosquitto_pub -h localhost -t vehicle/location -m \"%s\"",
29             locations[i]);
30
31         system(cmd);
32
33         printf("Location Sent: %s\n", locations[i]);
34
35         i++;
36
37         if(i >= 10)
38             i = 0;
39
40         Sleep(6000);
41     }
42
43     return 0;
44 }
```

C Program (Publisher Operation)

- Generates random GPS coordinates periodically
- Formats data as a string (e.g., "12.9716,77.5946")
- Publishes data to MQTT topic via broker
- Repeats transmission every 5 seconds

```

receiver.py
1  import paho.mqtt.client as mqtt
2  import webbrowser
3
4  def on_message(client, userdata, msg):
5      data = msg.payload.decode()
6      print("Received:", data)
7
8      lat, lon = data.split(",")
9
10     # Set a zoom level, for example, 12 (more zoomed out)
11     zoom = 12
12
13     url = f"https://www.google.com/maps?q={lat},{lon}&z={zoom}"
14     print("Opening:", url)
15     webbrowser.open(url)
16
17 client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1)
18
19 client.on_message = on_message
20
21 client.connect("localhost", 1883, 60)
22
23 client.subscribe("vehicle/location")
24
25 print("Waiting for location...")
26
27 client.loop_forever()

```

Python Program (Subscriber Operation)

- Connects to MQTT broker
- Subscribes to GPS topic
- Receives coordinate messages in real time
- Converts received string into numeric latitude & longitude
- Updates map by plotting each new location point
- Refreshes every 5 seconds for live tracking

MQTT Broker Operation

- Acts as a message mediator
- Receives data from publisher
- Forwards it to all subscribed clients

➤ Observed Results

Publisher Terminal:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF

PS C:\Users\amitk\OneDrive\Desktop\GPS_TRACKER> gcc publisher.c -o publisher.exe
PS C:\Users\amitk\OneDrive\Desktop\GPS_TRACKER> .\publisher.exe
Location Sent: 12.9788,77.5996
Location Sent: 12.9763,77.5929
Location Sent: 12.9797,77.5907
Location Sent: 12.9507,77.5848
Location Sent: 12.9756,77.6066
Location Sent: 12.9987,77.5920
Location Sent: 13.0098,77.5511
█
```

Receiver Terminal:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF

PS C:\Users\amitk\OneDrive\Desktop\GPS_TRACKER> python receiver.py
C:\Users\amitk\OneDrive\Desktop\GPS_TRACKER\receiver.py:17: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1)
Waiting for location...
Received: 12.9797,77.5907
Opening: https://www.google.com/maps?q=12.9797,77.5907&z=12
Received: 12.9507,77.5848
Opening: https://www.google.com/maps?q=12.9507,77.5848&z=12
Received: 12.9756,77.6066
Opening: https://www.google.com/maps?q=12.9756,77.6066&z=12
Received: 12.9987,77.5920
Opening: https://www.google.com/maps?q=12.9987,77.5920&z=12
Received: 13.0098,77.5511
Opening: https://www.google.com/maps?q=13.0098,77.5511&z=12
Received: 12.8008,77.5773
```

➤ Observed Results

- The C program successfully generated and transmitted simulated GPS coordinates.
- The MQTT broker correctly routed messages between publisher and subscriber.
- The Python application received GPS data without delay.
- Each received coordinate was plotted on a map in real time.
- The map updated every 5 seconds, showing continuous movement of location points.
- The system successfully demonstrated a complete IoT-style data pipeline using communication.

➤ Conclusion

The experiment successfully demonstrated MQTT-based communication between a C publisher and a Python subscriber using a local broker. Random GPS coordinates generated in the C program were transmitted through MQTT topics and received by Python in real time. The received data was visualized on a dynamically updating map, refreshing every 5 seconds. This experiment helped in understanding publish-subscribe architecture, IoT communication protocols, and real-time geospatial data visualization techniques widely used in modern tracking systems.